AWS to MikroTik Site-to-Site VPN

Step-by-step record of how the tunnel was built and tested (Saka store lab, 8 October 2026)

1. Summary

An IPsec (IKEv2) tunnel was built between an AWS VPC and the Saka store MikroTik router using the **AWS Site-to-Site VPN** service with a virtual private gateway and static routing. The store LAN can reach EC2 private IPs over the tunnel (tested: 10 of 10 pings to 172.31.42.211, about 195-245 ms).

Item	Value
AWS region / VPC	N. Virginia (us-east-1) / default VPC 172.31.0.0/16
Store LAN	192.168.155.0/24 (router LAN IPs 192.168.155.10 and 192.168.155.5)
Store public IP (router WAN)	197.234.57.98/30 on interface WAN, gateway 197.234.57.97
Router	MikroTik RB2011UiAS, RouterOS 6.49.15
AWS tunnel 1 outside IP	52.206.126.204 (tunnel 2: 54.204.187.131, not configured)
Routing / IKE	Static routing, IKEv2, AES-256, SHA2-256, DH group 14
Test EC2 private IP	172.31.42.211

2. Before you start

- The store LAN and the VPC CIDRs must **not overlap** (here 192.168.155.0/24 vs 172.31.0.0/16).
- The router needs a **public IP on its WAN interface**. Confirm with `/ip address print`.
- RouterOS 6.49.15 supports IPsec but **not WireGuard** (needs RouterOS v7). That is why IPsec was used.
- Take a router backup first: `/system backup save name=before-vpn`.
- If the router has two ISPs, make sure traffic to AWS leaves through the WAN that owns the public IP (see section 4, step 3).

3. AWS side (VPC console)

Step 1: Create the customer gateway

- VPC > Customer gateways > Create customer gateway.
- Name: saka-store. IP address: the store public IP (197.234.57.98). BGP ASN: leave 65000 (ignored with static routing).
- Certificate and Device fields: leave blank.

Step 2: Create and attach the virtual private gateway

- VPC > Virtual private gateways > Create. Name: saka-vgw. ASN: Amazon default.
- Select it > Actions > Attach to VPC > choose the default VPC.
- Do **not** use "Site-to-Site VPN Concentrator". It needs a transit gateway and is a different feature.

Step 3: Create the VPN connection

Setting	Value
Name tag	saka-vpn
Target gateway type	Virtual private gateway (saka-vgw)
Customer gateway	Existing: saka-store
Routing options	Static, prefix 192.168.155.0/24
Local IPv4 network CIDR	192.168.155.0/24 (store side)

Setting	Value
Remote IPv4 network CIDR	172.31.0.0/16 (AWS side)
Tunnel 1 > Edit options	Phase 1 and 2 encryption AES256, integrity SHA2-256, DH group 14, IKE version ikev2 only, startup action Start
Pre-shared key / inside CIDR	Leave as Generated by Amazon

Setting the local and remote CIDRs makes AWS use the same traffic selectors as the router policy. Wait about 5-10 minutes until the state changes from Pending to **Available**.

Step 4: Route propagation

- VPC > Route tables > the main route table of the default VPC > Route propagation tab > Edit.
- Tick the virtual private gateway and save. The Routes tab then shows 192.168.155.0/24 with the VGW as target (Propagated: Yes).
- Remove any older static route for 192.168.155.0/24 that points to an instance.

Step 5: Download the configuration

- Select the VPN > Download configuration. Vendor Generic (or Mikrotik) and IKE version **ikev2**.
- From the file, take only two values for IPsec Tunnel #1: the **Virtual Private Gateway outside IP** and the **pre-shared key**.
- Do not paste the sample commands. They use sha1, aes-128 and modp1024, which do not match the tunnel options chosen above.

4. MikroTik side (Winbox > New Terminal)

Replace the placeholders: **AWS_T1_IP** = tunnel 1 outside IP (52.206.126.204) and **PSK** = the tunnel 1 pre-shared key. Never store the PSK in documents or chats. Long lines below are wrapped with a trailing backslash, which RouterOS accepts as a continuation.

Step 1: Backup and checks

```
/system backup save name=before-vpn
/ip address print
```

Confirm the public IP is on the WAN interface and note the LAN IP to use as the ping source.

Step 2: Check the default route

```
/ip route print where dst-address=0.0.0.0/0
```

The active route (flag A) must go through the gateway of the WAN that holds the public IP (197.234.57.97).

Step 3: Pin the AWS endpoint to the correct WAN

```
/ip route add dst-address=AWS_T1_IP/32 gateway=197.234.57.97 comment="AWS VPN tunnel 1"
```

Prevents the tunnel traffic from switching to the second ISP if its default route becomes active.

Step 4: IPsec configuration

```
/ip ipsec profile add name=aws-prof enc-algorithm=aes-256 hash-algorithm=sha256 dh-group=modp2048 \
    lifetime=8h nat-traversal=no
/ip ipsec proposal add name=aws-prop auth-algorithms=sha256 enc-algorithms=aes-256-cbc \
    pfs-group=modp2048 lifetime=1h
/ip ipsec peer add name=aws-t1 address=AWS_T1_IP/32 local-address=197.234.57.98 profile=aws-prof \
    exchange-mode=ike2
/ip ipsec identity add peer=aws-t1 auth-method=pre-shared-key secret="PSK"
/ip ipsec policy add src-address=192.168.155.0/24 dst-address=172.31.0.0/16 \
    sa-src-address=197.234.57.98 sa-dst-address=AWS_T1_IP peer=aws-t1 tunnel=yes proposal=aws-prop \
    action=encrypt
```

Step 5: NAT exemption and firewall

```
/ip firewall nat add chain=srcnat action=accept src-address=192.168.155.0/24 \
    dst-address=172.31.0.0/16 place-before=0
/ip firewall filter add chain=input action=accept protocol=udp dst-port=500,4500 \
    src-address=AWS_T1_IP place-before=0
/ip firewall filter add chain=input action=accept protocol=ipsec-esp src-address=AWS_T1_IP \
    place-before=0
/ip firewall filter add chain=forward action=accept src-address=192.168.155.0/24 \
    dst-address=172.31.0.0/16 place-before=0
/ip firewall filter add chain=forward action=accept src-address=172.31.0.0/16 \
    dst-address=192.168.155.0/24 place-before=0
```

The NAT accept rule must be above the masquerade rule so tunnel traffic is not translated. If a place-before=0 line errors because the chain is empty, run it without that option.

5. Testing

From the router (use the LAN IP as source, not an IP the router does not own):

```
/ping 172.31.42.211 src-address=192.168.155.10
/ip ipsec active-peers print
/ip ipsec installed-sa print
```

In AWS, open the VPN connection > Tunnel details. **Tunnel 1 should show UP.** Tunnel 2 shows DOWN because it is not configured.

From EC2 to the store (reverse direction):

```
ping 192.168.155.10        # router LAN IP
ping 192.168.155.x         # a device on the store LAN
```

To ping the router itself from AWS, allow ICMP in the input chain:

```
/ip firewall filter add chain=input action=accept protocol=icmp src-address=172.31.0.0/16 \
    place-before=0
```

Also test from a normal store PC (ping and SSH to the EC2 private IP). Windows PCs block ping by default, so enable ICMPv4 echo or test with another device.

6. Troubleshooting

Symptom	Check
Tunnel stays DOWN	PSK, AWS tunnel IP and IKE version. Run `/log print where topics~"ipsec"`. Confirm UDP 500/4500 and ESP are allowed in the input chain.
Phase 1 fails	Router profile must match AWS options: aes-256, sha256, modp2048 (DH 14), IKEv2.
Tunnel UP, no traffic	NAT accept rule above masquerade, forward rules, route propagation in AWS, security group, and policy subnets matching the local/remote CIDRs.
EC2 cannot reach store	Device gateway must be the MikroTik, device firewall, and router input rule for ICMP if pinging the router.
Tunnel drops randomly	Second ISP default route taking over. Keep the pinned /32 route to the AWS endpoint.
Ping source error	Use an IP that exists on the router LAN (192.168.155.10), not 192.168.155.1 if the router does not own it.

7. Security and cleanup

- **Rotate the PSK** if it was ever shared in chat or email: VPN connection > Actions > Modify VPN tunnel options.
- Only tunnel 1 is configured, so there is no redundancy. For production configure tunnel 2 as well (note RouterOS 6 does not do active/active).
- Restrict the EC2 security groups to the store CIDR (192.168.155.0/24) and the ports actually needed, instead of all traffic.
- **Cost:** a Site-to-Site VPN connection is billed per hour plus data transfer. Delete it when the demo is finished.

Cleanup order

- Delete the VPN connection (saka-vpn).
- Detach and delete the virtual private gateway, then delete the customer gateway.
- Disable route propagation, release unused Elastic IPs and stop test EC2 instances.
- On the router remove the ipsec policy, identity, peer, proposal, profile, the NAT and firewall rules added, and the pinned route; or restore `before-vpn`.

8. Alternative: WireGuard on EC2

WireGuard on an EC2 gateway is cheaper and simpler but is a single point of failure that you must maintain, and it needs **RouterOS v7** on the MikroTik. It was not used here because the router runs 6.49.15. Upgrade only after a backup and outside trading hours.